University of Moratuwa
Department of Electronic and Telecommunication Engineering



Resource-Constrained CNNs for Traffic-Sign Classification

EN3150 – Pattern Recognition
Assignment 03

Group PixelPulse

Index number	Name
230197M	S.B. Garusinghe
230513G	R.H.G.S.R. Rajapaksha
230471A	M.V.N. Pelige
230495B	H.W.D. Prabharsha

19 October 2026

Contents

1 Data preparation **1**

1.1 Dataset and partitioning 1

1.2 Preprocessing and augmentation 2

1.3 Partition consistency and evaluation scope 2

2 Custom architecture design **2**

2.1 Model A: standard CNN 2

2.2 Model B: depthwise separable CNN 3

2.3 Computation and the effect of separable convolutions 4

2.4 Hardware-aware activation choices 4

3 Optimizer selection and tuning **4**

3.1 Controlled optimizer comparison on Model B 4

3.2 Effect of momentum 5

4 Custom model training and evaluation **6**

4.1 Training configuration and available runs 6

4.2 Model B training curves 6

4.3 Test results and confusion matrix 7

4.4 Model A and Model B resource comparison 8

5 Fine-tuning lightweight pretrained networks **8**

5.1 Network adaptation 8

5.2 Training behaviour 9

5.3 Test metrics and model storage 10

5.4 Recorded computational cost 12

6 Final comparison and trade-off discussion **12**

6.1 Accuracy versus memory 12

6.2 Computation and deployment constraints 13

6.3 Advantages and limitations of each approach 13

6.4 Conclusion 13

A Experimental record **14**

1 Data preparation

We used the German Traffic Sign Recognition Benchmark (GTSRB) to study the balance between classification accuracy and model cost. The task has 43 classes, including speed limits, warning signs and direction signs. We implemented a standard CNN (Model A), a depthwise separable CNN (Model B), and fine-tuned MobileNetV2 and EfficientNet-B0. Model B contains 54,251 trainable parameters, satisfying the 100,000-parameter limit.

1.1 Dataset and partitioning

The Kaggle dataset contains 39,209 original training images and 12,630 original test images. We combined these into 51,839 labelled images and applied a stratified 70/15/15 split with random seed 42. Thus, the test results in this report refer to our new partition, not to the official GTSRB test benchmark.

Table 1: Dataset partition sizes recorded in all three notebooks.

Subset	Images	Proportion
Training	36,287	70%
Validation	7,776	15%
Test	7,776	15%
Total	51,839	100%

Class frequencies are unequal. In the combined data, class 2 contains 3,000 images, whereas classes 0, 19 and 37 each contain 270. Stratification preserves class proportions approximately, and we report macro precision and recall alongside accuracy so that small classes also contribute equally to the evaluation.



Figure 1: Sample GTSRB images before preprocessing, with class labels and original dimensions.

1.2 Preprocessing and augmentation

For the custom models, we resized and padded RGB images to 32×32 using bilinear interpolation. Training-set channel statistics were

$$\boldsymbol{\mu} = (0.3219, 0.2955, 0.3048), \qquad \boldsymbol{\sigma} = (0.2742, 0.2621, 0.2689).$$

After conversion to tensors, each channel was normalised as $(x - \mu)/\sigma$. Training augmentation used rotations within $\pm 10^\circ$ and colour jitter with brightness 0.2, contrast 0.2 and saturation 0.1. Validation and test images received resizing, padding and normalisation only. Batch size was 64.

For MobileNetV2 and EfficientNet-B0, we used 64×64 RGB images and ImageNet normalisation: mean $(0.485, 0.456, 0.406)$ and standard deviation $(0.229, 0.224, 0.225)$. Their training transforms used random affine rotation of $\pm 10^\circ$, translation up to 8%, scale from 0.90 to 1.10, and colour jitter of 0.15, 0.15 and 0.10 for brightness, contrast and saturation. Evaluation used deterministic resizing and normalisation. Neither pipeline used horizontal flips, which could change the meaning of direction signs. Both resolutions satisfy the assignment's maximum of 64×64.

1.3 Partition consistency and evaluation scope

The runs use the same dataset size, stratification, seed and split proportions, but identical image membership across notebooks is not established. In `code_v3`, splitting follows the concatenated CSV order; `code_v4` sorts sample paths before splitting; and `code_v5` enumerates training folders before appending test rows. Both fine-tuning outputs say that a new split manifest was created. A fixed seed does not guarantee identical partitions when the input order changes.

Consequently, the results support a comparison of the recorded configurations, but not a strictly controlled comparison on verified identical test images. Reusing one path-based manifest across all models is necessary to satisfy the exact same-split requirement. The image-level split also does not separate sign tracks, so related views may appear in different subsets. This limits conclusions about performance on entirely new signs.

2 Custom architecture design

2.1 Model A: standard CNN

We first defined a LeNet-style baseline with two 5×5 convolutions (3 to 6 and 6 to 16 channels), two 2×2 max-pooling operations and fully connected layers $400 \to 120 \to 84 \to 43$. It contains 64,811 trainable parameters. The revised standard architecture, `ImprovedCNN`, adds a third convolutional stage and is used as Model A in the architectural comparisons below.

Each Model A convolution has a 3×3 kernel, stride 1, padding 1 and a bias. ReLU and 2×2 max pooling follow each convolution. The classifier contains a 128-unit hidden layer and a 43-unit output layer.

Table 2: Model A: revised standard CNN at 32×32 input. Shapes use channels $\times$ height $\times$ width.

Operation	Output shape	Parameters	MACs
Conv $3 \to 16$, ReLU	$16 \times 32 \times 32$	448	442,368
MaxPool 2×2	$16 \times 16 \times 16$	0	0
Conv $16 \to 32$, ReLU	$32 \times 16 \times 16$	4,640	1,179,648
MaxPool 2×2	$32 \times 8 \times 8$	0	0
Conv $32 \to 64$, ReLU	$64 \times 8 \times 8$	18,496	1,179,648
MaxPool 2×2	$64 \times 4 \times 4$	0	0
Flatten, FC $1024 \to 128$, ReLU	128	131,200	131,072
FC $128 \to 43$	43	5,547	5,504
Total		**160,331**	**2,938,240**

A biased convolution has $k^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}$ parameters, while a linear layer has $n_{\text{in}} n_{\text{out}} + n_{\text{out}}$. Therefore,

$$P_A = 448 + 4,640 + 18,496 + 131,200 + 5,547 = 160,331.$$

The first fully connected layer accounts for about 81.8% of Model A's parameters. Its large weight storage is a direct consequence of flattening the feature map.

2.2 Model B: depthwise separable CNN

Model B begins with a standard 3×3 stem convolution, followed by four depthwise separable blocks. Each block applies a channel-wise 3×3 convolution and then a 1×1 pointwise convolution. Batch normalisation and ReLU6 follow both operations. All convolutions are bias-free; the batch-normalisation layers supply trainable scale and shift. Pooling follows the first three separable blocks. Global average pooling and dropout with probability 0.25 precede the final linear classifier.

Table 3: Model B architecture; convolution stride is 1 and 3×3 padding is 1. DW denotes depthwise and PW denotes pointwise convolution.

Operation	Output shape	Parameters	MACs
Stem: Conv $3 \to 32$, BN, ReLU6	$32 \times 32 \times 32$	928	884,736
Block 1: DW 32, PW $32 \to 64$	$64 \times 32 \times 32$	2,528	2,392,064
MaxPool 2×2	$64 \times 16 \times 16$	0	0
Block 2: DW 64, PW $64 \to 128$	$128 \times 16 \times 16$	9,152	2,244,608
MaxPool 2×2	$128 \times 8 \times 8$	0	0
Block 3: DW 128, PW $128 \to 128$	$128 \times 8 \times 8$	18,048	1,122,304
MaxPool 2×2	$128 \times 4 \times 4$	0	0
Block 4: DW 128, PW $128 \to 128$	$128 \times 4 \times 4$	18,048	280,576
Global average pool, dropout, FC $128 \to 43$	43	5,547	5,504
Total		**54,251**	**6,929,792**

The parameter total can also be checked by component:

$$P_B = \underbrace{864}_{\text{stem weights}} + \underbrace{3,168}_{\text{depthwise weights}} + \underbrace{43,008}_{\text{pointwise weights}} + \underbrace{1,664}_{\text{BN scale/shift}} + \underbrace{5,547}_{\text{classifier}} = 54,251.$$

Batch-normalisation running means and variances are buffers rather than trainable parameters. Global average pooling reduces the final feature map to 128 values, avoiding a large flatten-to-dense layer.

2.3 Computation and the effect of separable convolutions

For one input image, convolutional MACs are counted as

$$\text{MACs} = H_{\text{out}} W_{\text{out}} C_{\text{out}} \frac{C_{\text{in}}}{g} k^2,$$

where g is the group count. A linear layer contributes $n_{\text{in}} n_{\text{out}}$ MACs. These counts exclude bias additions, batch normalisation, nonlinearities and pooling; they are operation estimates, not latency measurements.

For equal input/output dimensions, a standard 3×3 convolution uses $9C_{\text{in}}C_{\text{out}}$ weights, whereas a separable convolution uses $9C_{\text{in}} + C_{\text{in}}C_{\text{out}}$. Their weight and convolutional MAC ratio is

$$\frac{\text{separable}}{\text{standard}} = \frac{1}{C_{\text{out}}} + \frac{1}{9}.$$

Replacing Model B's four separable blocks by standard convolutions with the same channel layout gives an analytical reference with 394,443 parameters and 50,435,456 MACs. Model B reduces both by approximately 7.3× relative to that matched-width reference.

This reference is different from our actual Model A. Model A uses fewer channels and earlier spatial reduction, so its 2.94 million MACs are lower than Model B's 6.93 million. Model B has 66.2% fewer parameters than Model A, but approximately 2.36× as many counted MACs. Depthwise separability reduces the cost of a given convolution; it does not guarantee that an independently designed network has fewer operations overall.

2.4 Hardware-aware activation choices

Model A uses $\text{ReLU}(x) = \max(0, x)$ and Model B uses $\text{ReLU6}(x) = \min(\max(0, x), 6)$. Both require comparisons rather than exponential functions. ReLU6 bounds the activation range, which can help quantisation, although quantised accuracy was not measured. At inference, batch-normalisation statistics can be folded into preceding convolutions. The output is a vector of logits; selecting its largest element gives the predicted class without evaluating softmax. Cross-entropy training still involves the corresponding log-softmax computation.

3 Optimizer selection and tuning

3.1 Controlled optimizer comparison on Model B

We compared Adam, SGD and SGD with momentum using freshly initialised Model B instances and seed 42. The runs used the same custom-model data loaders, cross-entropy loss and batch size 64. The routine is configured for 20 epochs and retains the highest-validation-accuracy checkpoint. The saved output contains 20 epochs for Adam and SGD, and 17 for SGD with momentum; the latter is reported only over its recorded interval.

Table 4: Optimizer settings and recorded validation results for Model B.

Metric	Adam	SGD	SGD + momentum
Learning rate	0.001	0.01	0.01
Momentum setting	$(\beta_1, \beta_2) = (0.9, 0.999)$	0	0.9
Recorded epochs	20	20	17
Validation accuracy at epoch 1	82.20%	17.08%	56.30%
Validation accuracy at epoch 5	99.46%	42.61%	98.75%
First epoch reaching 95%	2	19	3
First epoch reaching 99%	4	Not reached	6
Best validation accuracy	99.73%	96.54%	99.70%
Mean training time/epoch	212.77 s	211.44 s	212.92 s

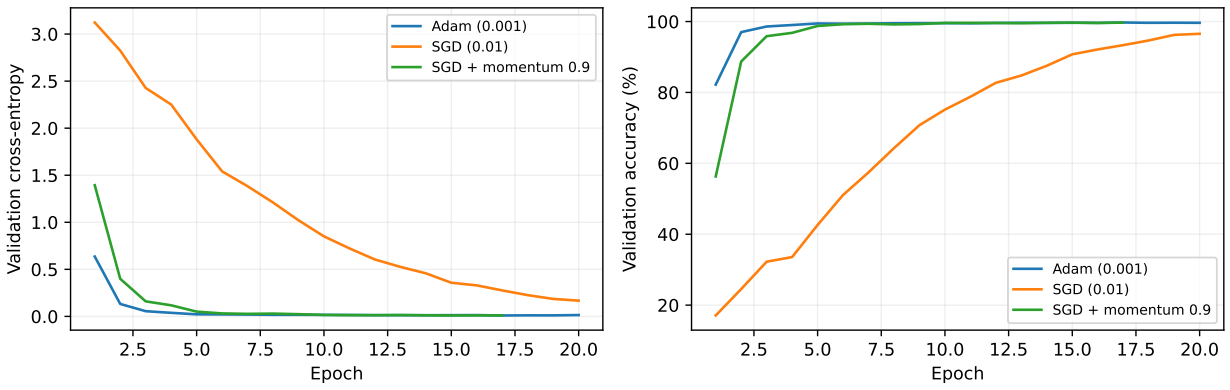


Figure 2: Validation loss and accuracy from the saved optimizer-comparison logs. The momentum curve ends at the last recorded epoch.

We selected Adam with learning rate 10^{-3} for the custom models because it reached high validation accuracy quickly in this comparison. Adam maintains first- and second-moment estimates of gradients and adapts the update magnitude for each parameter. Its extra optimizer state increases training memory, but is not needed for inference. This experiment compares selected configurations rather than an exhaustive learning-rate search. Plain SGD's lower score at learning rate 0.01 does not establish that SGD is inherently inferior.

3.2 Effect of momentum

For gradient g_t, the momentum update can be written as

$$v_t = \mu v_{t-1} + g_t, \qquad w_{t+1} = w_t - \eta v_t.$$

Gradients pointing consistently in one direction accumulate, while alternating components partially cancel. For a constant gradient, the steady-state update magnitude is proportional to $\eta/(1-\mu)$; at $\mu = 0.9$ this is ten times the plain-SGD magnitude at the same learning rate. This is an explanation of the update dynamics, not a claim of tenfold faster training.

At epoch 5, momentum raised validation accuracy from 42.61% to 98.75%. It reached 95% at epoch 3 instead of epoch 19. Adam and momentum reached similar best validation accuracies, with a difference of only 0.03 percentage points. One run per setting is insufficient to establish statistical superiority. Large momentum can also delay responses to changing gradient directions and cause overshoot; only momentum 0 and 0.9 were evaluated here.

4 Custom model training and evaluation

4.1 Training configuration and available runs

Both Model A implementations contain 20-epoch Adam training loops at learning rate 0.001. The revised Model A loop selects the best validation checkpoint. However, all Model A training, plotting and evaluation cells in the saved notebook have empty outputs. Therefore, its measured accuracy, precision, recall, confusion matrix, loss curves and training time cannot be recovered from the experimental record. The architectural quantities in Section 2 are calculated directly from its layer definitions.

Model B has a separate 20-epoch training log and test evaluation, in addition to the optimizer experiment. Its test evaluation uses the final model from that loop, rather than the best checkpoint from the later Adam comparison. The first recorded epoch already has 98.69% training accuracy, so we treat this as a recorded training interval without claiming that it demonstrates convergence from a fresh initialisation. The clean from-scratch Adam convergence evidence is the optimizer run in Section 3.

4.2 Model B training curves

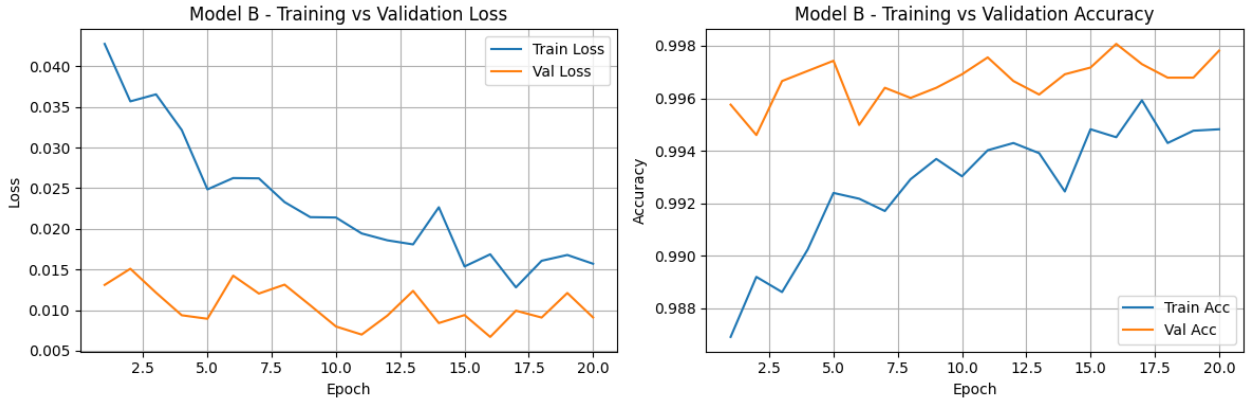


Figure 3: Model B's saved 20-epoch training interval associated with the subsequent test evaluation.

Table 5: Selected epochs from Model B's test-associated training interval.

Epoch	Training loss	Validation loss	Training accuracy	Validation accuracy
1	0.0428	0.0131	98.69%	99.58%
5	0.0249	0.0089	99.24%	99.74%
10	0.0214	0.0080	99.30%	99.69%
16	0.0169	0.0067	99.45%	99.81%
20	0.0157	0.0091	99.48%	99.78%

Training loss falls overall, while validation loss remains low with small fluctuations. The lower validation loss is consistent with training-only augmentation and dropout. Training metrics also average predictions across changing weights within an epoch. The curves do not show sustained validation deterioration over this interval, but they cannot by themselves rule out memorisation or optimistic evaluation caused by related images across splits.

4.3 Test results and confusion matrix

For each class, precision is $TP/(TP + FP)$ and recall is $TP/(TP + FN)$. Macro averaging gives all classes equal weight; weighted averaging uses each class's support. In single-label multiclass classification, weighted recall equals overall accuracy.

Table 6: Model B test results on 7,776 images.

Metric	Value
Cross-entropy loss	0.0082
Accuracy	99.7171%
Correct / incorrect predictions	7,754 / 22
Weighted precision	0.9972
Weighted recall	0.9972
Weighted F1	0.9972
Macro precision	0.9960
Macro recall	0.9964

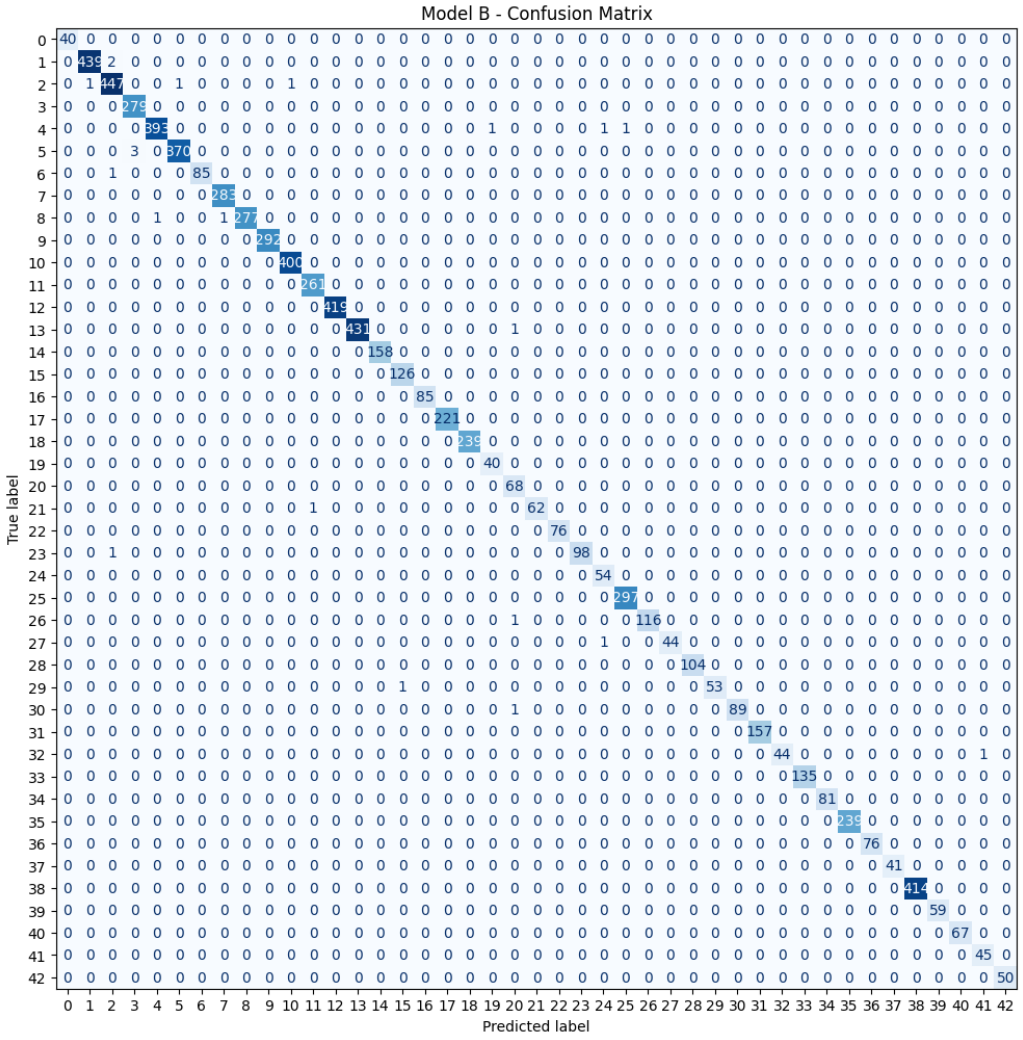


Figure 4: Model B test confusion matrix. Rows are true classes and columns are predicted classes; the colour scale represents image counts.

The matrix is strongly diagonal. Macro precision and recall are slightly lower than their weighted counterparts, indicating that equal weighting of the less frequent classes gives a modestly lower score. With only 22 errors, the evaluation is near its ceiling and small accuracy differences should be interpreted cautiously.

4.4 Model A and Model B resource comparison

Table 7: Calculated resource quantities for the implemented custom architectures. Float32 storage excludes buffers and serialization overhead.

Metric	Model A	Model B
Trainable parameters	160,331	54,251
Float32 parameter storage (KiB)	626.29	211.92
Convolution and linear MACs/image	2,938,240	6,929,792
Input resolution	32×32	32×32

Model B's saved state dictionary occupies 235.56 KiB (241.21 decimal KB), including batch-normalisation buffers and serialization overhead. Its from-scratch Adam optimizer run averages 212.77 seconds per training epoch, excluding validation. The earlier report identifies this as a CPU run; the corresponding device output is not preserved in `code_v3`. The separate test-associated training loop does not record timings.

For Model A, the estimated float32 parameter payload is 626.29 KiB (641.32 decimal KB). No serialized checkpoint size or training timer output is available, and no test result is recorded. Thus the supplied experiments do not complete the measured Model A–Model B accuracy and timing comparison required in Question 4. We report this limitation explicitly instead of assigning Model A an unmeasured result.

The main supported trade-off is that Model B substantially reduces weight storage through separable blocks and global average pooling, while its wider early feature maps increase its total MAC count relative to Model A. Low parameter count, low arithmetic cost and low measured latency are distinct design goals.

5 Fine-tuning lightweight pretrained networks

5.1 Network adaptation

We fine-tuned torchvision MobileNetV2 in `code_v5` and EfficientNet-B0 in `code_v4`. Both were initialised with their default ImageNet pretrained weights, and the final classifier was replaced by a $1280 \to 43$ linear layer. The replacement layer has

$$1280 \times 43 + 43 = 55,083$$

trainable parameters. All backbone weights remained trainable.

MobileNetV2 uses inverted residual blocks with depthwise convolution and linear bottlenecks. EfficientNet-B0 also uses mobile inverted bottleneck blocks, together with squeeze-and-excitation and SiLU activations. These designs reduce the cost of large conventional convolutional networks, but their parameter counts still greatly exceed our 100,000-parameter custom-model budget.

Table 8: Fine-tuning settings used for both pretrained models.

Setting	Value
Input / batch size	64×64 RGB / 64
Optimizer	AdamW, learning rate 10^{-4}, weight decay 10^{-4}
Loss	Cross-entropy
Learning-rate schedule	Cosine annealing, $T_{\max} = 15$
Training duration	15 epochs; early-stopping patience 6
Precision	CUDA fp16 autocast with gradient scaling
Model selection	First checkpoint achieving the highest validation accuracy
Data loading	Two workers; validation and test without augmentation

The lower learning rate allows pretrained features to adapt gradually. AdamW applies decoupled weight decay, and cosine annealing reduces the learning rate during fine-tuning. Both runs completed 15 epochs without early stopping. The 20-epoch minimum in Question 4 applies to the custom networks; Question 5 specifies no corresponding minimum.

5.2 Training behaviour

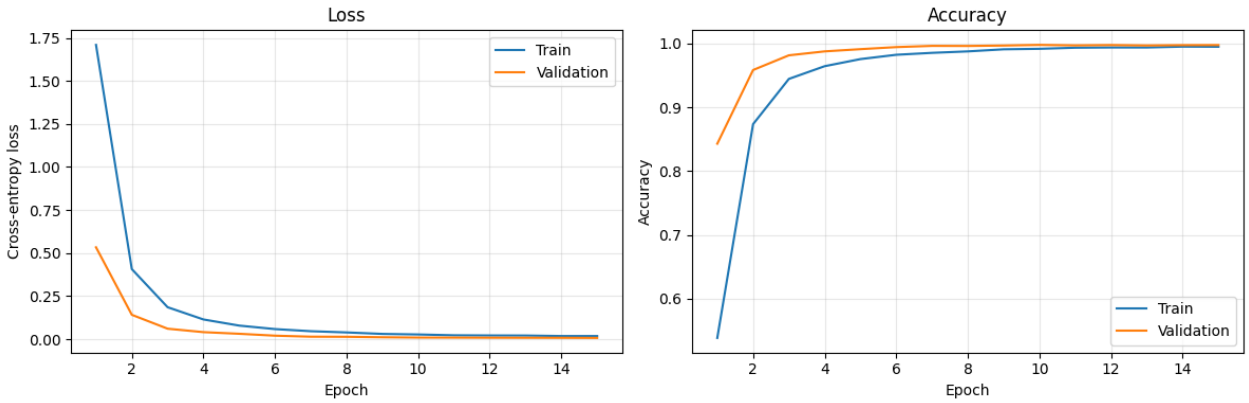


Figure 5: MobileNetV2 training and validation curves across 15 epochs.

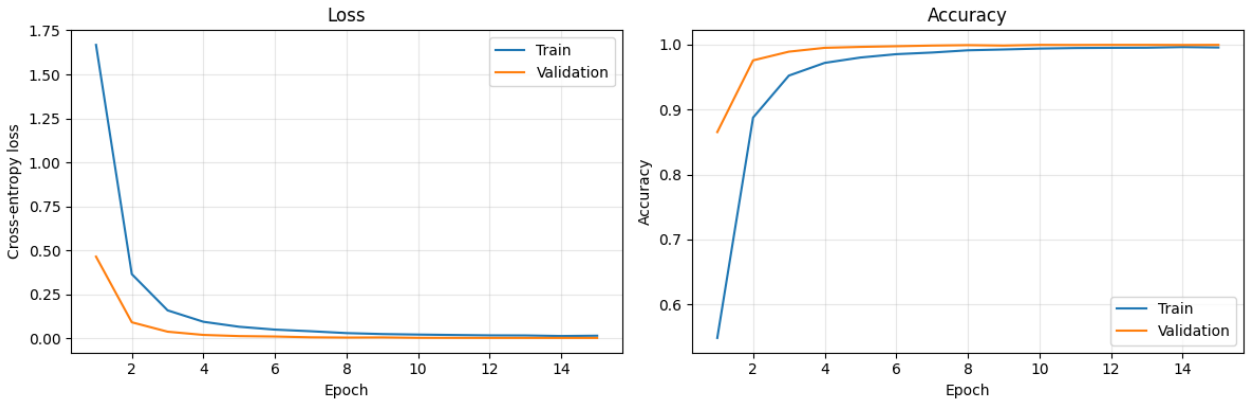


Figure 6: EfficientNet-B0 training and validation curves across 15 epochs.

Table 9: Fine-tuning progress and checkpoint selection.

Metric	MobileNetV2	EfficientNet-B0
Epoch 1 training loss	1.7090	1.6678
Epoch 1 validation accuracy	84.2850%	86.5098%
Epoch 15 training loss	0.0179	0.0157
Epoch 15 validation loss	0.0070	0.0032
Best validation accuracy	99.7685%	99.9100%
Selected epoch	10	10

Both models improve rapidly in the first few epochs and then approach a plateau. EfficientNet-B0 reaches a higher best validation accuracy, although this does not translate into higher recorded test accuracy. Later epochs sometimes have lower validation loss without higher accuracy because cross-entropy also reflects prediction confidence. The selection rule uses accuracy, so the first maximum at epoch 10 is retained. Augmentation, training-mode regularisation and within-epoch weight updates help explain why validation metrics can look better than training metrics.

5.3 Test metrics and model storage

Table 10: Test results and model storage from the saved fine-tuning outputs. All parameters are trainable.

Metric	MobileNetV2	EfficientNet-B0
Total parameters	2,278,955	4,062,631
Saved checkpoint (MiB)	8.928	15.783
Saved checkpoint (decimal MB, approximate)	9.362	16.550
Test accuracy	99.8200%	99.8200%
Correct / incorrect predictions	7,762 / 14	7,762 / 14
Macro precision	0.998699	0.998865
Macro recall	0.998278	0.999090
Macro F1	0.998483	0.998974
Weighted precision	0.9982	0.9982
Weighted recall	0.9982	0.9982
Weighted F1	0.9982	0.9982

The notebooks label their size values as MB but divide byte counts by 1024^2, which gives MiB. We distinguish the units here. Mixed-precision computation during fine-tuning does not imply that the saved state dictionary contains only half-precision weights.

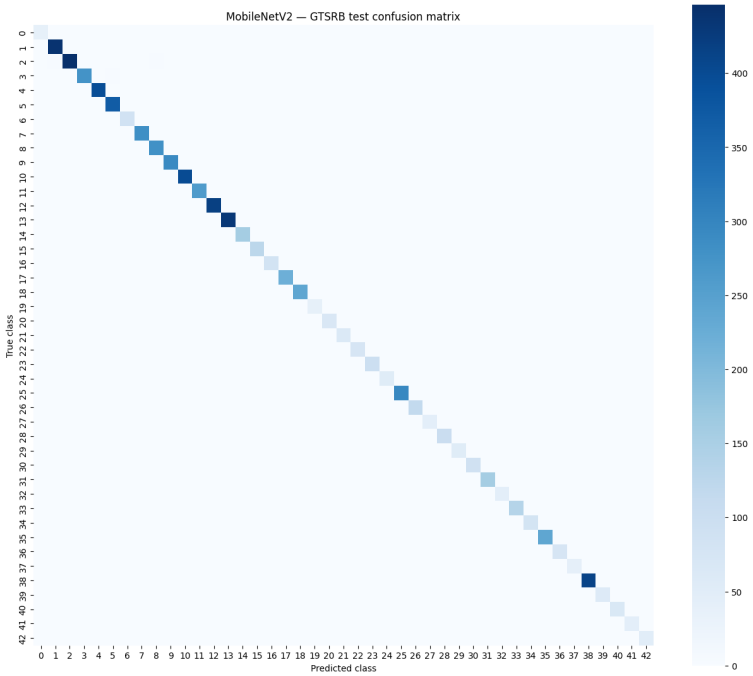


Figure 7: MobileNetV2 test confusion matrix for the 43 classes. Colour intensity reflects counts; individual errors are small relative to the diagonal.

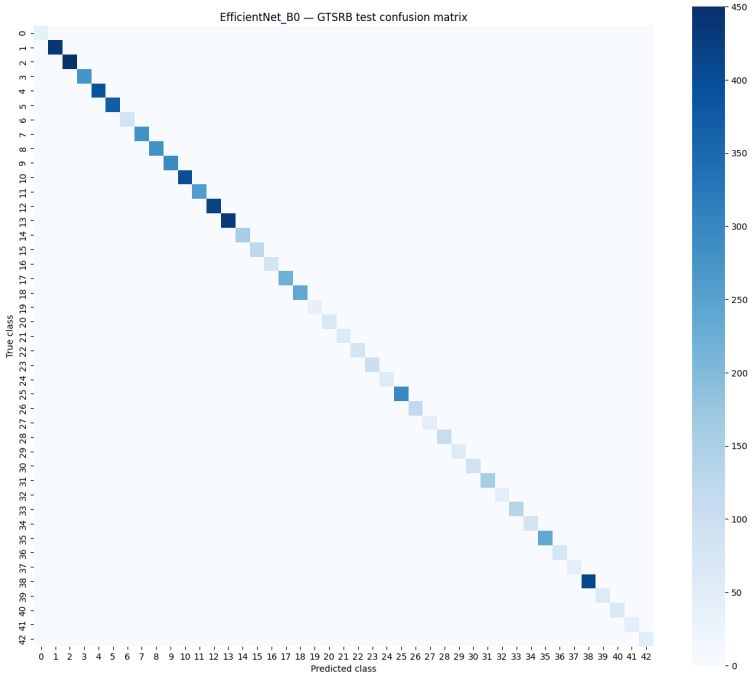


Figure 8: EfficientNet-B0 test confusion matrix for the 43 classes.

EfficientNet-B0 has 14 errors across seven true classes: classes 1, 3, 4, 5, 7, 13 and 38. Twelve are speed-limit errors. MobileNetV2 also has 14 errors, but they are distributed across nine true classes: 1, 2, 3, 5, 13, 19, 25, 26 and 30. Its lowest reported class recall is 0.975 for class 19, corresponding to one error among 40 images. These counts are recovered from the per-class recall and support outputs. They describe errors by true class, not the destination classes of those errors.

The models therefore have equal overall test accuracy but different per-class behaviour. EfficientNet-B0's slightly higher macro scores reflect a different allocation of errors across class sizes. The split-order differences in Section 1 prevent attributing this difference entirely to the architecture.

5.4 Recorded computational cost

Table 11: Timing measurements in the fine-tuning notebooks.

Metric	MobileNetV2	EfficientNet-B0
Mean training + validation time/epoch	49.64 s	64.34 s
Batched inference time/image	0.1504 ms	0.3083 ms
Inference batch size	64	64
Recorded device	CUDA	CUDA (Tesla T4)

Inference timing skips three warm-up batches and averages the next 20. It times the forward pass after device transfer and uses GPU synchronisation; it excludes image decoding and host-to-device transfer. Dividing batch time by image count gives an amortised throughput measure, not single-image end-to-end latency.

The recorded MobileNetV2 inference value is about half the EfficientNet-B0 value, but the MobileNetV2 output does not identify the GPU model. We therefore do not interpret the ratio as a controlled hardware benchmark. Likewise, these CUDA timings cannot be compared directly with the earlier report's CPU timing for Model B. Convolutional MAC counts for the pretrained models were not recorded, so no unverified scaled-resolution estimate is substituted for them.

6 Final comparison and trade-off discussion

6.1 Accuracy versus memory

Table 12: Comparison of Model B with the two fine-tuned networks. Results are from the recorded configurations, with unverified cross-model split identity.

Metric	Model B	MobileNetV2	EfficientNet-B0
Input resolution	32×32	64×64	64×64
Parameters	54,251	2,278,955	4,062,631
Parameters relative to B	1.00×	42.01×	74.89×
Checkpoint size (MiB)	0.2300	8.928	15.783
Test accuracy	99.7171%	99.8200%	99.8200%
Test errors	22	14	14
Macro precision	0.9960	0.998699	0.998865
Macro recall	0.9964	0.998278	0.999090

Both pretrained configurations have a recorded accuracy advantage of approximately 0.103 percentage points over Model B, corresponding numerically to eight fewer errors in equally sized test sets. This is a small absolute difference in return for much greater weight storage. MobileNetV2's checkpoint is about 38.8× larger than Model B's, and EfficientNet-B0's is about 68.6× larger. Their input images also contain four times as many pixels, so the comparison includes changes in resolution and augmentation as well as architecture and pretraining.

Model B is the strongest choice among these configurations when the main constraint is weight storage. Its compact size comes from channel-wise filtering, pointwise mixing and global average pooling. Pretrained networks offer reusable features and a standard architecture with little manual

design, but their million-parameter backbones exceed a sub-100k budget even after replacing the classifier.

6.2 Computation and deployment constraints

Weight size is only one part of deployment memory. Activations, temporary operator buffers and the inference runtime also occupy RAM. For example, Model B's first 64-channel feature map at 32×32 contains 65,536 values, or 256 KiB in float32, before accounting for other live tensors. A 235.56 KiB checkpoint therefore does not establish that inference fits within the same RAM budget.

Model B requires 6.93 million convolutional/linear MACs per image. Its depthwise operations save arithmetic compared with a matched-width standard network, but operator support and memory movement affect real speed. Our narrower Model A has lower calculated MACs despite higher parameter storage. This is why a deployment decision should consider weight memory, peak activation memory and measured latency separately.

As a theoretical storage estimate, one byte per parameter would reduce Model B's weight payload to about 53.0 KiB. This excludes quantisation scales, buffers and metadata, and no quantised model was evaluated. Quantisation could also reduce the pretrained models' weight payloads, but their parameter-count ratios would remain unchanged. Accuracy after compression and latency on the intended device remain unmeasured.

6.3 Advantages and limitations of each approach

A custom network allows direct control of channel counts, pooling locations, activation functions and classifier size. This makes it possible to meet a strict parameter limit, but selecting and training the architecture requires experimentation. Model B achieves high recorded accuracy with a small weight payload; its remaining limitations are the unmeasured target-device performance and the absence of a verified common split across all runs.

Fine-tuning gives a useful starting representation and rapid convergence in both recorded SOTA runs. MobileNetV2 uses substantially fewer parameters and less checkpoint storage than EfficientNet-B0 while reaching the same test accuracy here. EfficientNet-B0 reaches higher validation accuracy and slightly higher macro test metrics, but the additional memory cannot be justified by an overall test-accuracy gain in these results. Neither model has been benchmarked on the intended edge device.

6.4 Conclusion

Our depthwise separable Model B meets the assignment's parameter constraint with 54,251 trainable parameters and reaches 99.7171% recorded test accuracy. MobileNetV2 and EfficientNet-B0 both reach 99.8200%, with much larger checkpoints. The main observed benefit of the custom network is storage efficiency; the main benefit of the pretrained configurations is high accuracy with a reusable architecture and a pretrained starting point.

The architectural analysis also shows that fewer parameters do not necessarily mean fewer operations: Model A has higher weight storage but lower calculated MACs than Model B. The absent Model A execution results and the unverified equality of split membership limit the experimental comparison. These limitations do not change the reported measurements, but they prevent claiming a complete controlled evaluation of all four networks.

A Experimental record

The report uses `code_v3.ipynb` for the custom models and optimizer comparison, `code_v4.ipynb` for EfficientNet-B0, and `code_v5.ipynb` for MobileNetV2. Figures are extracted from saved notebook outputs, except the optimizer plot, which is reconstructed from printed epoch logs. Parameter totals, Model A MACs, storage conversions and comparison ratios are arithmetic calculations from the architecture definitions and recorded values. No additional training run is included.

GitHub repository

The assignment code is available at:
https://github.com/DeminiPrabharsha/EN3150-Assignment-03/